4

Retrieving Data

In this chapter, you'll learn how to use the `SELECT` statement to retrieve one or more columns of data from a table.

The `SELECT` Statement

As explained in Chapter 1, "Understanding SQL," SQL statements are made up of plain English terms. These terms are called *keywords*, and every SQL statement is made up of one or more keywords. The SQL statement you'll probably use most frequently is the `SELECT` statement. Its purpose is to retrieve information from one or more tables.

To use `SELECT` to retrieve table data, you must, at a minimum, specify two pieces of information: what you want to select and from where you want to select it.

Tip

Use MySQL Workbench to Follow Along As noted previously, you are strongly encouraged to try each and every example as you work through this book. Actually, you should also experiment and tweak the SQL yourself. The more you do so, the more comfortable you'll become with MySQL syntax. Use MySQL Workbench to do this, as explained in Chapter 3, "Working with MySQL."

Retrieving Individual Columns

We'll start with a simple `SQL SELECT` statement, as follows:

► `Input`

```
SELECT prod_name
FROM products;
```

► `Analysis`

This statement uses the `SELECT` statement to retrieve a single column called `prod_name` from the `products` table. The desired column name is specified right after the `SELECT`

keyword, and the `FROM` keyword specifies the name of the table from which to retrieve the data. This statement provides the following output:

Output

```
+------------------ +
| prod_name         |
+------------------ +
|.5 ton anvil       |
|1 ton anvil        |
|2 ton anvil        |
|Oil can            |
|Fuses              |
|Sling              |
|TNT (1 stick)      |
|TNT (5 sticks)     |
|Bird seed          |
|Carrots            |
|Safe               |
|Detonator          |
|JetPack 1000       |
|JetPack 2000       |
+------------------ +
```

Note

Unsorted Data If you tried this query yourself (you did, right?), you might have discovered that the data was displayed in a different order than shown here. If this is the case, don't worry; it is working exactly as it is supposed to. If query results are not explicitly sorted (we'll get to that in the next chapter), data will be returned in no order of any significance. It might be the order in which the data was added to the table, but it might not. As long as your query returned the same number of rows as shown here, then it is working.

A simple `SELECT` statement like the one just shown returns all the rows in a table. Data is not filtered (to retrieve a subset of the results), nor is it sorted. We'll discuss these topics in the next few chapters.

Note

Terminating Statements Multiple SQL statements must be separated by semicolons (; characters). MySQL (like most other DBMSs) does not require that a semicolon be used after a single statement. You can always add a semicolon if you wish. It'll do no harm, even if it isn't needed.

If you are using the `mysql` command-line client, the semicolon is always needed (as explained in Chapter 2, "Introducing MySQL").

Note

SQL Statements and Case It is important to note that SQL statements are not case-sensitive, so `SELECT` is the same as `select` or `Select`. Many SQL developers find that using uppercase for all SQL keywords and lowercase for column and table names makes code easier to read and debug. (That's the formatting style used in this book.)

However, be aware that while the SQL language is not case-sensitive, identifiers (the names of databases, tables, and columns) might be.

As a best practice, pick a case convention and use it consistently.

Tip

Use of White Space All extra white space within a SQL statement is ignored when the statement is processed. SQL statements can be specified on one long line or broken up over many lines. The following statements are thus functionally identical:

```
SELECT prod_name FROM products;

SELECT
prod_name
FROM
products;

        SELECT
           prod_name
              FROM
                 products;
```

Most SQL developers find that breaking up statements over multiple lines makes the statements easier to read and debug.

White space (spaces, tabs, line breaks, etc.) is ignored when SQL statements are processed. So how does the DBMS know when your statement is finished? That's what the ; at the end does.

Retrieving Multiple Columns

To retrieve multiple columns from a table, you use the same `SELECT` statement. The only difference is that you must specify multiple column names after the `SELECT` keyword, and you must separate the columns with commas.

Tip

Take Care with Commas When selecting multiple columns, be sure to include a comma after each column name except the last one. Including a comma after the last name will generate an error.

The following `SELECT` statement retrieves three columns from the `products` table:

► Input

```
SELECT prod_id, prod_name, prod_price
FROM products;
```

► Analysis

Just as in the previous example, this statement uses the `SELECT` statement to retrieve data from the `products` table. In this example, three column names are specified, separated by commas. The output from this statement is as follows:

► Output

```
+---------+---------------+------------+
| prod_id | prod_name     | prod_price |
+---------+---------------+------------+
| ANV01   | .5 ton anvil  | 5.99       |
| ANV02   | 1 ton anvil   | 9.99       |
| ANV03   | 2 ton anvil   | 14.99      |
| OL1     | Oil can       | 8.99       |
| FU1     | Fuses         | 3.42       |
| SLING   | Sling         | 4.49       |
| TNT1    | TNT (1 stick) | 2.50       |
| TNT2    | TNT (5 sticks)| 10.00      |
| FB      | Bird seed     | 10.00      |
| FC      | Carrots       | 2.50       |
| SAFE    | Safe          | 50.00      |
| DTNTR   | Detonator     | 13.00      |
| JP1000  | JetPack 1000  | 35.00      |
| JP2000  | JetPack 2000  | 55.00      |
+---------+---------------+------------+
```

Note

Presentation of Data SQL statements typically return raw, unformatted data. Data formatting is a presentation issue, not a retrieval issue. Therefore, presentation (for example, alignment and displaying the price values as currency amounts with the currency symbol and commas) is typically specified in the application that displays the data. Actual raw retrieved data (without application-provided formatting) is rarely displayed as is.

Retrieving All Columns

In addition to being able to specify one or more columns, `SELECT` statements can also request all columns without having to list them individually. This is done using the asterisk (*) wildcard character in lieu of actual column names, as follows:

► Input

```
SELECT *
FROM products;
```

► Analysis

When a wildcard (*) is specified, all the columns in the table are returned. Columns are returned in the order in which they appear in the table definition. However, changes to table schemas (such as adding and removing columns) may cause ordering changes.

Caution

Using Wildcards As a rule, you are better off not using the * wildcard unless you really do need every column in the table. Even though using wildcards might save you the time and effort needed to list the desired columns explicitly, retrieving unnecessary columns usually slows down the performance of your retrieval and your application.

Tip

Retrieving Unknown Columns There is one big advantage to using wildcards. As you do not explicitly specify column names (because the asterisk retrieves every column), it is possible to retrieve columns whose names are unknown.

Retrieving Distinct Rows

As you have seen, `SELECT` returns all matched rows. But what if you did not want every occurrence of every value? For example, suppose you wanted the vendor ID of every vendor with products in your `products` table and used the following:

► Input

```
SELECT vend_id
FROM products;
```

► Output

```
+---------+
| vend_id |
+---------+
|    1001 |
|    1001 |
|    1001 |
```

```
|    1002 |
|    1002 |
|    1003 |
|    1003 |
|    1003 |
|    1003 |
|    1003 |
|    1003 |
|    1003 |
|    1005 |
|    1005 |
+---------+
```

► Analysis

This `SELECT` statement returns 14 rows because there are 14 products listed in the `products` table. But those 14 products are actually sold by just 4 vendors. So how could you retrieve a list of distinct vendor values?

The solution is to use the `DISTINCT` keyword which, as its name implies, instructs MySQL to return only distinct values:

► Input

```
SELECT DISTINCT vend_id
FROM products;
```

► Analysis

`SELECT DISTINCT vend_id` tells MySQL to return only distinct (unique) `vend_id` rows, and so only 4 rows are returned, as shown in the following output. When you use the `DISTINCT` keyword, you must place it directly in front of the column names.

► Output

```
+--------- +
| vend_id  |
+--------- +
|     1001 |
|     1002 |
|     1003 |
|     1005 |
+--------- +
```

Caution

Can't Be Partially `DISTINCT` The `DISTINCT` keyword applies to all columns, not just the one it precedes. If you were to specify `SELECT DISTINCT vend_id, prod_price`, all rows would be retrieved unless *both* of the specified columns were distinct.

Limiting Results

The SELECT statement returns all matched rows—and possibly every row—in the specified table. To return just the first row or rows, use the LIMIT clause. Here is an example:

► Input

```
SELECT prod_name
FROM products
LIMIT 5;
```

► Analysis

This example uses the SELECT statement to retrieve a single column. LIMIT 5 instructs MySQL to return no more than 5 rows. The output from this statement is as follows:

► Output

```
+---------------- +
| prod_name       |
+---------------- +
| .5 ton anvil    |
| 1 ton anvil     |
| 2 ton anvil     |
| Oil can         |
| Fuses           |
+---------------- +
```

To get the next 5 rows, specify both where to start and the number of rows to retrieve, like this:

► Input

```
SELECT prod_name
FROM products
LIMIT 5,5;
```

► Analysis

LIMIT 5,5 instructs MySQL to return 5 rows, starting from row 5. The first number is where to start, and the second is the number of rows to retrieve. The output from this statement is as follows:

► Output

```
+---------------- +
| prod_name       |
+---------------- +
| Sling           |
| TNT (1 stick)   |
| TNT (5 sticks)  |
| Bird seed       |
| Carrots         |
+---------------- +
```

So, LIMIT with one value specified always starts from the first row, and the specified number is the number of rows to return. LIMIT with two values specified can start from wherever that first value tells it to.

Caution

Row 0 The first row retrieved is row 0, not row 1. Therefore, `LIMIT 1,1` will retrieve the second row, not the first one.

Note

When There Aren't Enough Rows The number of rows to retrieve specified in LIMIT is the *maximum* number to retrieve. If there aren't enough rows (for example, if you specify `LIMIT 10,5`, but there are only 13 rows), MySQL returns as many as it can.

Tip

Using `LIMIT OFFSET` Does `LIMIT 3,4` mean 3 rows starting from row 4 or 4 rows starting from row 3? As you just learned, it means 4 rows starting from row 3, but it is a bit ambiguous.

For this reason, MySQL supports an alternative syntax for LIMIT. `LIMIT 4 OFFSET 3` means to get 4 rows starting from row 3, just like `LIMIT 3,4`. `LIMIT OFFSET` tends not to be used much, and the reason I am mentioning it here is so that you'll know what it is if you see it in the wild.

Using Fully Qualified Table Names

The SQL examples used thus far have referred to columns by using just the column names. It is also possible to refer to columns by using fully qualified names (that is, using both the table and column names). Look at this example:

Input

```
SELECT products.prod_name
FROM products;
```

Analysis

This SQL statement is functionally identical to the very first one used in this chapter, but here a fully qualified column name is specified. It explicitly states that the `prod_name` column it's referring to is in the `products` table.

Table names, too, may be fully qualified, as shown here:

Input

```
SELECT products.prod_name
FROM crashcourse.products;
```

Analysis

Once again, this statement is functionally identical to the one just used (assuming, of course, that the `products` table is indeed in the `crashcourse` database).

There are situations when fully qualified names are required, as you will see in later chapters. For now, it is worth noting this syntax so you'll know what it is if you run across it.

Using Comments

As you have seen, MySQL statements are instructions that are processed by the MySQL DBMS. But what if you want to include text that you do not want to be processed and executed? Why would you ever want to do that? Here are a few reasons:

- The SQL statements we've been using here are all very short and very simple. But, as your SQL statements grow in length and complexity, you'll want to include descriptive comments (for your own future reference or for whoever must work on the project next). These comments need to be embedded in the SQL scripts, but they are obviously not intended for actual DBMS processing.
- The same is true for introductory text at the top of a SQL file, which you'll often want to contain a description and notes and perhaps even programmer contact information.
- Another important use for comments is to temporarily stop SQL code from being executed. If you are working with a long SQL statement and want to test just part of it, you can *comment out* some of the code so that the DBMS sees it as comments and ignores it.

MySQL supports several forms of comment syntax. We'll start with inline comments:

Input

```
SELECT prod_name    -- this is a comment
FROM Products;
```

Analysis

Comments may be embedded inline using `--` (two hyphens). Any text on the same line that is after the `--` is considered comment text, making this a good option for describing columns in a `CREATE TABLE` statement, for example.

Here is another form of inline comment:

Input

```
# This is a comment
SELECT prod_name
FROM Products;
```

Analysis

`#` can be used anywhere in your SQL. Anything that follows this character is comment text, so `#` at the start of a line makes the entire line a comment.

You can also create multiline comments and comments that stop and start anywhere within the script:

Input

```
/* SELECT prod_name, vend_id
FROM Products; */
SELECT prod_name
FROM Products;
```

Analysis

`/*` starts a comment, and `*/` ends it. Anything between `/*` and `*/` is comment text. This type of comment is often used to *comment out* code, as shown in this example. Here, two `SELECT` statements are defined, but the first one won't execute because it has been commented out.

Summary

In this chapter, you learned how to use the SQL `SELECT` statement to retrieve a single table column, multiple table columns, and all the columns in a table. You also learned how (and why) to comment your SQL code. Next, you'll learn how to sort the retrieved data.

Challenges

1. Write a SQL statement to retrieve every customer ID (`cust_id`) from the `customers` table.
2. The `OrderItems` table contains every item ordered (some of which were ordered multiple times). Write a SQL statement to retrieve a list of the products (`prod_id`) ordered (not every order, just a unique list of products). Here's a hint: You should end up with nine unique rows displayed.
3. Write a SQL statement that retrieves all columns from the `customers` table and an alternate `SELECT` statement that retrieves just the customer IDs. Use comments to comment out one `SELECT` statement so you can run the other one. (And, of course, test both statements.)